ARPE TITLE

Anonymous authors
Paper under double-blind review

ABSTRACT

...

Neural Circuits as a Design Principle

[Gregoire]: This part serves as introduction, biblio, background etc. for the rest of the document.

[Gregoire]: Add note on magno and parvo pathways (in vision) as an example of parallel pathways with different dynamics and functions. Magno is fast and feedback connection typically target magno pathways to meet up with slower parvo feedforward and recurrent pathways at a target area, implementing predictive and modulatory processing. Also, FB is typically long ranged (spatially) while recurrent is local - spectral vs spatial parameterisation of orthogonal convolution. We can model magno as skip connections, in which case the overall pathway reduces to a simple recurrence, and SSM solve both recurrent and feedback connection for the price of none (!). If we want to clearly separate FB and REC, we can use two layers, one parameterised spectrally, one spatially. This would cost a bit more but would be more biologically plausible and more interpretable.

1 INTRODUCTION

Present the overall project, motivations, initial goals, how it fits into my program, etc. Present the structure of the report.

2 BIOLOGICAL AND COMPUTATIONAL MOTIVATION

Brief overview of computational neuroscience, biological neural circuits and modeling efforts, and links to machine learning history.

3 FORMALIZING NEURAL CIRCUITS

[Gregoire]: TODO : references

Abstract Neural Circuits. Neural computation in the brain is organized around circuits: structured assemblies of neurons whose collective dynamics implement specific transformations of information. Rather than isolated units or uniform layers, cortical processing emerges from patterned connectivity, where the nature of the computation is largely determined by how neurons are connected across space and time. In this work, we adopt a deliberately abstract notion of neural circuits, intended to bridge biological and artificial systems without committing to anatomical realism or specific implementation details. At this level of abstraction, a neural circuit is characterized not by the biophysical properties of individual neurons, but by the organization of its connections and the resulting dynamical behavior. This perspective allows a unified treatment of biological circuits and artificial neural networks, viewing both as instances of structured dynamical systems.

Definition 1 (Circuit)**.** *A circuit is defined by a directed graph (V, E) along with a set of dynamical equations governing the evolution of each unit $v \in V$ state over time. The dependencies between these equations are determined by the graph structure, while the specific form of the equations can vary widely.*

Let V be the set of units (neurons) in the circuit, and let $E \subseteq V \times V$ be the set of directed connections between them. Each connection $(u, v) \in E$ has an associated weight $w_{uv} \in \mathbb{R}$. The state of the circuit at time t is given by a vector

$h(t) \in \mathbb{R}^{|V|}$, where $h_v(t)$ is the activity of unit v at time t. The evolution of the circuit's state can either be expressed in continuous time through differential equations:

$$\frac{\partial h}{\partial t}(t) = F(h(t)) \tag{1}$$

or in discrete time:

$$h_{t+1} = \overline{F}(h_t) \tag{2}$$

where F and $\overline{F}$ are functions that define the dynamics of the circuit.

[Thomas]: Consider reframing this definition from a dynamical systems / comp neuro perspective rather than graph-theoretic. As written, the graph (V, E) and weights w_{uv} are introduced but F is left completely general, so the graph is formally redundant until you specialize to the linear case below. Suggestion: start from the state equation $h'(t) = F(h(t), x(t))$ and let the connectivity *emerge* as a structural property of F—a connection $j \to i$ exists iff $\partial F_i / \partial h_j \not\equiv 0$. This way the graph is derived from the dynamics. Benefits: (1) connects directly to the SSM linearization in Part II (F linear $\Rightarrow h' = Ah + Bx$, sparsity of A is the graph); (2) stability analysis follows naturally from eigenvalues of the Jacobian; (3) better framing for ICLR audience who think in RNNs and state equations. Keep the graph perspective for the connectivity taxonomy (FF/FB/lateral, circulant, block-sparse)—that is where it genuinely helps.

Input and Output Nodes. To prevent this system's dynamic to depend only on its initial state, we can introduce the notion of input nodes, which inject external signal into the circuit, conditioning its internal dynamics. Input nodes differ from other units in that their activity is determined externally, rather than by the circuit's internal dynamics : $h_{\text{input}}(t) = x(t)$ where $x(t)$ is the external input signal at time t. Some nodes might also play the role of output nodes, whose activity is read out by other circuits or systems. These output nodes are readily modelled by selecting a subset of V. Input and output nodes provide an interface with other circuits or sensory/motor systems. Figure 1 provides an example of basic circuit with an input node a, recurrent connections on nodes b and c and feedback connections from d to b. Its set of differential equations is given below :

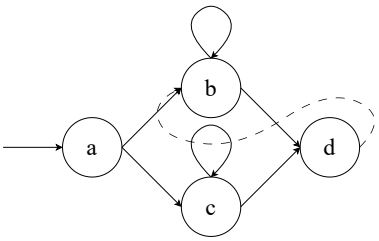

Figure 1: Example of circuit.

$$h_a(t) = x(t), \qquad h'_b(t) = (h_a + h_b + h_d)(t),$$
$$h'_c(t) = (h_a + h_c)(t), \qquad h'_d(t) = (h_b + h_c)(t)$$

Circuit Scales. At the finest level, a circuit's units may correspond to individual neurons, while at coarser levels they may represent populations of neurons, other circuits, or entire brain regions. Fine-grained circuits may be embedded within larger-scale networks, with interactions occurring across multiple spatial and temporal scales. They often exhibit block-structured connectivity patterns, both in biological and artificial systems, where blocks are commonly referred to as layers due to their hierarchical arrangement with respect to information flow from input units. These blocks can be *contracted* into single nodes, whose edges with other blocks represent the connectivity pattern between the units of the two blocks.

Connectivity structure. The most common models both for biological and artificial neural systems define F as a linear transformation followed by a pointwise nonlinearity, as in Equations (3) and (4). On the biological side, this choice is motivated by the observation that neurons integrate their inputs linearly through synaptic potentials before generating spikes via a nonlinear activation function. On the artificial side, this formulation has proven effective for learning complex functions through gradient-based optimization and GPU acceleration.

$$\frac{\partial h}{\partial t}(t) = \sigma\left(Wh(t) + b\right) \tag{3}$$

$$h_{t+1} = \sigma\left(\overline{W}h_t + \overline{b}\right) \tag{4}$$

where $W \in \mathbb{R}^{|V| \times |V|}$ is the weight matrix with $W_{uv} = w_{uv}$ if $(u, v) \in E$ and 0 otherwise, $b \in \mathbb{R}^{|V|}$ is a bias vector, and σ is a pointwise nonlinearity. However, other types of interactions are possible, such as multiplicative (or divisive) interactions (used in the KV circuits of attention layers, or [TODO: biological example]), which can then generate gating mechanisms (GRU, [TODO: biological example]), etc.

However, modeling W is often impractical as $|V|$ grows large, due to the quadratic growth in the number of parameters. Therefore, structured connectivity patterns are often imposed to reduce the number of parameters. Incorporating prior knowledge about the task or data can guide the choice of connectivity structure, improving learning efficiency and generalization by providing inductive biases aligned with the underlying problem.

The most wide spread type of structure is that of block sparsity provided by a layered structure, where connections are only allowed between specific layers. Let $L_0, ..., L_n$ provide the layered structure, with layers sorted from input to output, and neurons ordered based on their layer membership. Directed bloc connectivity can be separated into *feedforward*, *feedback* and *recurrent* connections. When units from layer L_i project onto units in layer L_j, we refer to *feedforward* connections when $i < j$, *feedback* when $i > j$ and *recurrent* when $i = j$ (see Figure 2). The latter are also commonly referred to as *lateral* or *horizontal* connections based on author and context. In the present work, we distinguish between *self*- recurrent connections to and from the same unit, and *lateral* connections between arbitrary units of the same layer in order to avoid confusion between the special and general cases.

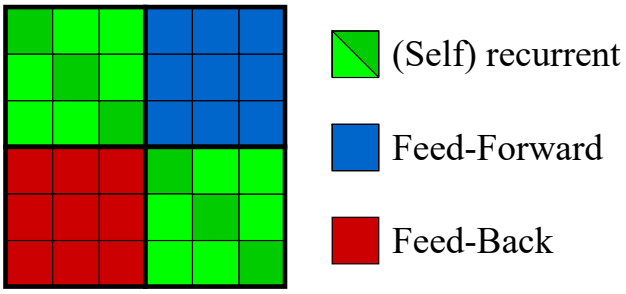

Figure 2: Illustration of adjacency matrix, with connections colored by type. Nodes are grouped by layer and sorted from input to output.

We can further refine the connectivity by imposing structure on blocks of the adjacency matrix. The most common type of such structure is that of *local* connections, where connections are spatially constrained to a spatial neighborhood. This translates to diagonal sparsity in the corresponding block (Figure 3 left). This structure is famously salient in the visual cortex and in convolutional neural networks (CNN). Another common structure is that of repeating circuits (**?**), translating in *circulant* connectivity (Figure 3 right). Such structure naturally models repeating microcircuits and implies translational equivariance. It can also be found in the visual cortex and in CNNs where it is typically combined with *locality* leading to very efficient operationalization through convolution kernels.

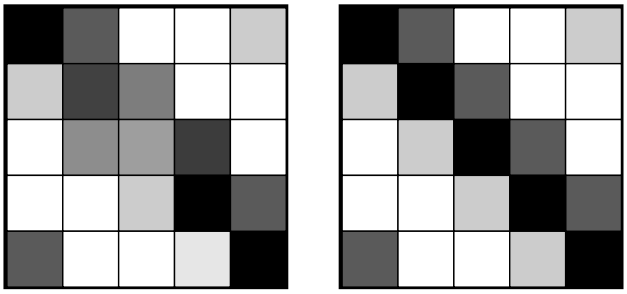

Figure 3: **Left :** illustration of a block containing horizontal connections. **Right :** a circulant matrix, imposing translational invariance on top of locality.

[Gregoire]: TODO : delete the paragraph below. Put it, along with other practical commentaries, in a dedicated section.

Flesh vs. Silicon Circuits. Circuits play a fundamental role in both biological and artificial neural systems. In the brain, specialised circuits underlie sensory processing, motor control, memory, and cognition. Examples of biological circuits include cortical microcircuits with recurrent and lateral connections, thalamocortical loops, and hippocampal networks involved in memory. [TODO: expand on biological circuits examples]. Figure 4 illustrates the whole circuit of the visual cortex, while Figure 5a shows [TODO : a sub circuit implementing some known function].

In artificial neural networks, circuit-like architectures such as recurrent networks, convolutional neural networks (CNN) with lateral connections, and attention mechanisms enable complex temporal and spatial processing capabilities.

Placeholder Placeholder Placeholder

(a) TODO (b) TODO (c) TODO

Figure 4: TODO

Architectures are often designed to mimic biological circuit connectivity patterns and functions. The term also refers to specific features of some architectures, such as the Key-Query (KQ) and Output-Value (OV) circuits in transformer attention mechanisms. Artificial settings also allow for easier studies of fine grained circuits and sub-circuits implementing specific functions. Examples of such sub-circuits are known to implement specific computations such as working memory in recurrent architectures, edge detection or higher level feature extraction in visual systems. In language models, circuits have been identified that implement specific linguistic functions such as syntactic processing, as well as higher level semantic functions. Figure 4 illustrates the full structure of a CNN mimicking the visual cortex, while Figure 5b shows an example of sub-circuit in GPT2 implementing indirect object identification, along with each component's role in this circuit. [Gregoire]: TODO : citations !

Placeholder Placeholder

(a) TODO (b) TODO

Figure 5: TODO

4 Neural Circuits in Brain Sciences

The formalism introduced in Section 3, despite its simplicity and generality, proves to be a powerful tool to model and analyse biological neural circuits. We review several key examples from the literature where this formalism has been successfully applied to model neural circuits in various brain areas and species, highlighting its versatility and effectiveness. We discuss how this formalism allows to capture essential features of neural dynamics, connectivity patterns, and functional roles of different brain regions, providing insights into the underlying mechanisms of neural computation.

Example Circuit 1. ...TODO...

Example Circuit 2. ...TODO...

The Visual System. The visual system is a prime example of a complex neural circuit that can be effectively modeled using the formalism introduced in Section 3. The visual system consists of multiple interconnected brain regions, including the retina, lateral geniculate nucleus (LGN), primary visual cortex (V1), and higher-order visual areas. Each of these regions can be represented as a set of neurons (nodes) with specific connectivity patterns (edges) that define the flow of visual information through the system. For instance, the retina acts as input nodes as it processes incoming light signals and transmits them to the LGN, which in turn relays the information to V1 for further processing. [Gregoire]: feedforward connections A are dense, while B are sparse and project to ... . Who are each of these structures ? The lateral and feedforward connections within V1 and between higher-order visual areas are structured *locally*, with a substantial amount of repeating circuit motifs, providing a *circulant* connectivity pattern that enables efficient processing of visual features such as edges, orientations, and motion. Feedback connections from higher to lower areas also play a crucial role in modulating visual processing based on context and attention. Feedback connections can typically provide *long-range* connections between distant spatial locations in the visual field, while lateral connections are more *local*, connecting nearby neurons within the same area. [Gregoire]: TODO : Complete this description a bit more. Include ventral vs dorsal pathways and so on. Include known fine circuits such as edge detectors, the local excitation vs global inhibition circuit that thomas described, etc. Fig. 6 illustrates a schematic representation of the visual system as a neural circuit, highlighting the key brain regions and their connectivity patterns.

Placeholder

Figure 6: Schematic representation of the visual system as a neural circuit. The retina processes incoming light signals and transmits them to the LGN, which relays the information to V1 for further processing. Feedforward connections (solid arrows) transmit information from lower to higher areas, while recurrent connections (dashed arrows) enable local processing within areas. Feedback connections (dotted arrows) modulate processing based on context and attention. The block connectivity patterns ([Gregoire]: **right**) reflect the functional roles of each brain region in the hierarchy of visual processing. Some blocks's specific structure are indicated in dedicated bubbles.

Delays and feedbacks. However, this formalism is not without limitations. The biology of neural circuits includes several more layers of complexity that are not captured by this simple framework. For instance, synaptic plasticity mechanisms allow neural circuits to adapt and learn from experience. Specific cell types, physiological properties such as axon myelination which affects signal propagation speed, along with neuromodulators and countless other biological factors are not explicitly modeled here. We briefly touch on one such aspect which will be of particular interest for **??**[Gregoire]: check that crefs to parts is not broken, which boils down to time delays in signal propagation. To model delays in our formalism, each edge $(u, v) \in E$ can be associated with a delay parameter $\tau_{uv} \geq 0$, representing the time it takes for a signal to propagate from neuron u to neuron v. The neuron-wise state update equation then becomes:

$$h_v(t) = F_v\left(\{h_u(t - \tau_{uv}) : (u, v) \in E\}\right) \tag{5}$$

Several factors influence these delays, including axon length and diameter, myelination, and synaptic transmission times. Understanding and modeling these delays is essential for accurately capturing the dynamics of neural circuits and their functional roles in information processing. An example of crucial functionality enabled by delays is the introduction of oscillatory dynamics in recurrent circuits, which are thought to play a role in various cognitive functions such as attention, binding and working memory. Another setting where delays play a crucial role is examplified in the visual system. For instance, magnocellular pathways are known for their rapid signal transmission, which is crucial for processing motion and temporal changes, even at the cost of coarse spatial resolution. In contrast, parvocellular pathways, which process fine spatial details and color, exhibit longer delays. Feedforwards account for slower parvocellular pathways, while feedback connections typically follow rapid magnocellular pathways, allowing for quick top-down modulation of sensory processing. This speed in feedback pathways alows for rapid contextual integration across a wide spatial area, while lateral connections within an area tend to be slower and more spatially constrained.

[Gregoire]: Small illustration for long range feedback vs local recurrence ?

5 Neural Circuits in Machine Learning

Overview of existing machine learning architectures that can be interpreted as neural circuits, highlighting their structure and function. State how most are purely feedforward, with recurrent architectures still marginal despite the recent SSM developpements, and feedback connections being even more rare and constrained to specific use cases such as predictive coding. Transformers can have a sort of time "recurrence" through positional encoding, but this isn't fit for long range sequences due to computational complexity. Plus, recurrence is quite artificial as it is added through attention masks and positional encoding, there is no hidden state that evolves over time. Each token is independent from the others, and it works thanks to full memory preservation. Recurrent and feedback architectures are not only more biologically plausible, but they also offer potential advantages in terms of efficiency, adaptability, interpretability and dynamics depending on the task.

6 Scope and Research Questions

We explore several aspects that naturally emerge from the above framework. First is architecture design grounded in neural circuits for a single modality (here, vision) (part II). Second, is the integration of unimodal circuits into a single multimodal one (part III and IV). Finally, in part V we explore the possibility of a universal neural circuit, akin to universal turing machines, that could simulate any other neural circuit, possibly under some constraints to make the problem tractable. This could serve as a foundation for a foundation model for brain science.

Constructing Better Vision Circuits

Part Outline

In this part, we explore how the circuit formalism of part I can guide the design of new architectures. Specifically, our goal is to introduce horizontal recurrent connections, where current SSMs only have self-recurrence.

7 Related Work

ConvNet, SSM, ConvS4, hGRU, etc. Also, dino is approximated by a recurrent architecture with just two layers (Fel et al).

8 ConvSSM Definition

State Space Model. The state space model (SSM) is a class of recurrent model with linear recurrent dynamics. Their most general formulation is that of continuous-time (Equation (6)), which can be discretised as in Equation (7), with transition matrices depending on the desired step size following Equation (9). Parameterizing transition matrices of the continuous time formulation allows to handle arbitrary sampling rates for free. Additionally, Mamba (**?**) takes advantage of this continuous time formulation and discretization step to derive a data-adaptive gating mechanism to give more or less importance to the present input.

$$h'(t) = Ah(t) + Bx(t) \quad (6a) \qquad h_t = \overline{A}h_{t-1} + \overline{B}x_t \quad (7a) \qquad \overline{\boldsymbol{K}} = (\boldsymbol{C}\overline{\boldsymbol{B}}, \boldsymbol{C}\overline{\boldsymbol{A}}\overline{\boldsymbol{B}}, \dots, \boldsymbol{C}\overline{\boldsymbol{A}}^k\overline{\boldsymbol{B}}, \dots) \quad (8a)$$

$$y(t) = Ch(t) \quad (6b) \qquad y_t = Ch_t \quad (7b) \qquad y = x * \overline{\boldsymbol{K}} \quad (8b)$$

Discretisation. Various rules can be used for discretisation, such as the zero-order hold (ZOH) whose solution is given in Equation (9).

$$\overline{A} = e^{\Delta \boldsymbol{A}} \qquad \overline{B} = (\Delta \boldsymbol{A})^{-1}(e^{\Delta \boldsymbol{A}} - \boldsymbol{I}) \cdot \Delta \boldsymbol{B} \tag{9}$$

where Δ is the time step.

Spatial Coupling. Rewriting Equation (6) to include horizontal connections yields Equation (10), with the corresponding discrete time formulations in Equation (11).

$$\frac{\partial h}{\partial t}(r,t) = K_h * h(r,t) + K_x * x(r,t) \quad (10a) \qquad h_t = \overline{K}_h * h_{t-1} + \overline{K}_x * x_t \quad (11a)$$

$$y(r,t) = K_y * h(r,t) \quad (10b) \qquad y_t = K_y * h_t \quad (11b)$$

However, discretization of the convolution kernel isn't as straightforward due to spatial coupling. Indeed, in order to be in the linear recurrence case, we need to view a whole image as a single vector $x \in \mathbb{R}^{(h \cdot w) \times c}$ and expand the convolution kernel into a circulant matrix $A \in \mathbb{R}^{(h \cdot w)^2 \times c^2}$. Now, following Equation (9), we can compute $\overline{A}$. However, $\overline{B}$ requires the inverse of A which typically has full spatial support, making it intractable in spatial domain.

By diagonalizing the circulant matrix in spectral domain, the set of spatially coupled partial differential equations (PDE) from Equation (10a) becomes a set of spectrally decoupled ordinary differential equations (ODE) (Equation (12a)) :

$$\frac{\partial \widehat{h}}{\partial t}(w,t) = \widehat{K}_h(w)\widehat{h}(w,t) + \widehat{K}_x(w)\widehat{x}(w,t) \quad (12a)$$

making Equation (9) applicable to every spatial frequency separately :

$$\overline{K}_h(w) = e^{\Delta \widehat{\boldsymbol{K}}_h(w)} \qquad \overline{K_x}(w) = (\Delta \widehat{\boldsymbol{K}}_h(w))^{-1}(e^{\Delta \widehat{\boldsymbol{K}}_h(w)} - \boldsymbol{I}) \cdot \Delta \widehat{\boldsymbol{K}}_x(w) \tag{13}$$

Stability. The continuous time ODEs imply exponential dynamics of the state over time. This dynamics are controlled by the eigenvalues λ_i^A of the transition matrix A ($\widehat{K}_h(w)$ in our case). The imaginary part controls the angular speed, while the real part controls the modulus' exponential growth. Thus, stability is achieved when

$$\mathrm{Re}(\lambda_i^A) \leq 0 \tag{14}$$

The corresponding discrete time condition is thus given by

$$|\lambda_i^{\overline{A}}| \leq 1 \tag{15}$$

State of the art SSM parameterisation enforces Equation (15) on A. They remark that in general it does not seem to work (which is expected as it is not the correct condition), but in the special case of HiPPO matrices, it happens to work well. Through a happy accident, these matrices can be decomposed in a normal plus low rank (NPLR) form with a normal component that satisfies the correct condition of Equation (14).

Link to Neural Circuits. As mentioned in **??**, the state space model implements *recurrent* connections through linear dynamics to make them parallelisable. State of the art approaches in vision make these *self-recurrent* only (**?**). Our approach adds *intra-recurrence* through *horizontal* connections for visual systems in such a way that preserves computational stability and complexity, while adding expressivity and biological plausibility.

9 ConvSSM Complexity

Recall that the goal is to implement the following recurrence :

$$h_{t+1} = \overline{K}_h \mathrm{A} * h_t + \overline{K}_x \mathrm{B} * I_{t+1} \tag{16}$$

where h_i is the hidden state at time step i, I_i is the input image - or feature map. $\overline{K}_h \mathrm{A}$ and $\overline{K}_x K_\mathrm{B}$ are convolution kernels. For simplicity, we will omit $\overline{K}_x K_\mathrm{B}$ in the remainder of this section. We show that computing the sequence of hidden states $(h_i)_{i \in [\![0,T-1]\!]}$ given a sequence of inputs $(I_i)_{i \in [\![0,T-1]\!]}$ can be seen as computing a scan operation.

We analyse the complexity of the parallel scan algorithm to show why convolutions in spatial domain make the complexity explode, and how doing convolutions in spectral domain preserves $O(\log T)$ complexity. We consider both the work complexity W, measuring the total amount of atomic operations to perform, and the time complexity under unlimited parallelization T_∞, measuring the longest sequence of atomic operations remaining after optimal parallelization.

We begin by addressing the main issue of the complexity explosion of convolutional SSMs in Sections 9.1 to 9.4. Our main result is Proposition 4, where we show that we can achieve the same work and time complexity as a regular linear SSM. We then tackle the question of reducing the constant to make it tractable, using standard tricks, adapted to our framework, in Section 9.5.

Mamba-style note. In this section, we consider transition matrices and transition kernels to be fixed over time. Note that it is entirely possible to have different weights at each time step simply by modifying the initial sequence of $(a_i)_{i \in I}$ defined below. In particular, following **?** one could parameterize Δ to depend on the present input x_i.

9.1 The parallel scan algorithm

We first describe the parallel scan operation in the most general way. This operation is the core of the SSM parallelization. Let $(a_i)_{i \in I} \in M^I$ be elements of a monoid $(M, +, 0)$, with $I = [\![0, T-1]\!]$. The parallel scan is an algorithm that computes $(s_i = \sum_{j=0}^{i} a_j)_i \in I$. This algorithm is parallelisable and requires associativity of the $+$ operator and a neutral element 0, hence the minimal requirement of a monoid structure. It proceeds in two sweeps mirroring each other. The first sweep proceeds in a tree like structure, with level l of the tree computing $a_k^{l+1} := a_{2k}^l + a_{2k+1}^l$, and initialised with $a_k^0 := a_k$. Once this upward sweep is over, the downward sweep computes mirror operations, with $b_1^0 := 0$ and $(b_{2k}^{l+1}, b_{2k+1}^{l+1}) = (b_k^l, b_k^l + a_{2k}^{\log_2(T)-(l+1)})$. The order is crucial here as $+$ is not required to be commutative. It is only required to be associative. In our application for SSM, it will not be commutative. We denote the parallel scan procedure as $\mathrm{Scan}((a_i)_{i \in I})$ or Scan for short. Figure 7 provides a visualisation of the entire procedure.

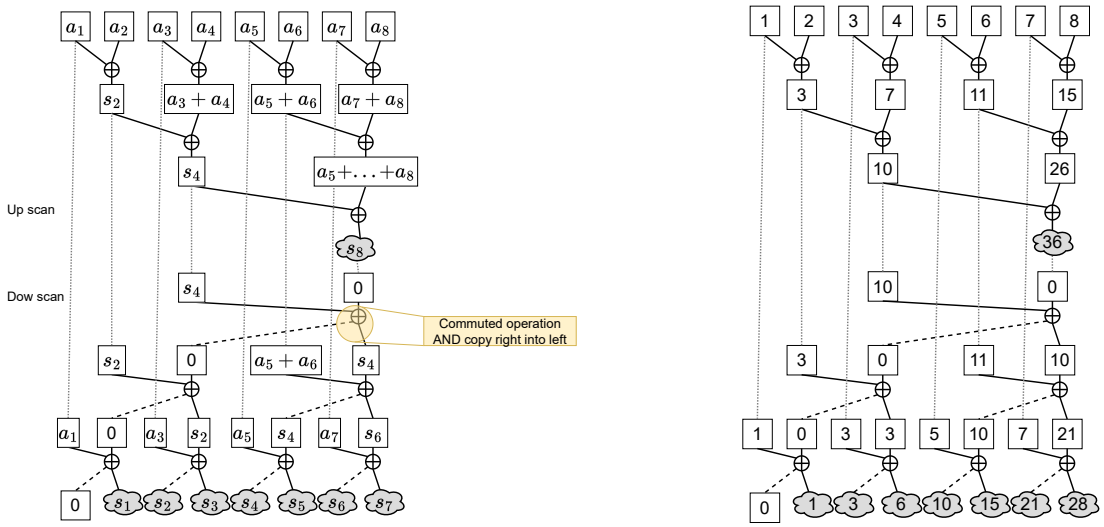

Figure 7: **Left:** parallel scan with a generic monoid. **Right:** illustration on the spetial case of natural integers.

The procedure described above yields $(\sum_{j=0}^{i-1} a_j)_{i \in I}$, with the cumulative sum being $a_0^{\log_2(T)}$ the intermediate result of the upward sweep. We analyse the work complexity W and the time complexity T_∞ of Scan in terms of floating point operations, where W counts the total number of operations needed and T_∞ counts the time complexity under unlimited parallelism.

Proposition 1 (Parallel Scan Complexity for $(\mathbb{N}, +)$)**.** *Let $(M, +) := (\mathbb{N}, +)$ be the monoid of natural integers equipped with addition. Let $(a_i)_{i \in I} \in \mathbb{N}^I$ be a sequence of integers.*

$$W(\text{Scan}) = 2(T - 1) \tag{17}$$

$$T_\infty(\text{Scan}) = 2 \log_2 T \tag{18}$$

Proof. At level l of the upward scan, $U_l = T/(2^{l+1})$ "+" operations are performed, each of them having a complexity of 1. There are $\log_2 T$ levels in total. The same goes on for the downward scan, with $D_l = T/(2^{\log_2(T)-l}) = 2^l$ operations at level l. Therefore, the work complexity of level l is U_l (resp. D_l) while its time complexity is 1 with unlimited parallelism. It follows that

$$W = 2 \sum_{l=0}^{\log_2(T)-1} 2^l = 2(T - 1) \tag{19}$$

$$T_\infty = 2 \sum_{l=0}^{\log_2(T)-1} 1 = 2 \log_2 T \tag{20}$$

□

9.2 The case of SSM

The case of SSMs is a little more complicated than that of real numbers. In SSMs, the recurrence is defined by Equation (16) with the slight modification that kernel convolution is now matrix multiplication and taking $B = I$. Thus, the goal is to compute in parallel the hidden states $(h_i = \sum_{j=0}^{i} \overline{A}^{i-j} x_j)_i \in I$, where x_i are vectors of dimension d and

$\overline{A}$ is a square matrix. This is almost but not quite what we need for the parallel scan. We define the following monoid:

$$M^{\text{SSM}} := \left\{a = (A, x) \mid A \in \mathcal{M}_d(\mathbb{R}),\ x \in \mathbb{R}^d\right\} \tag{21}$$

$$(A_a, x_a) +^{\text{SSM}} (A_b, x_b) := (A_b \odot A_a, A_b \cdot x_a + x_b) \tag{22}$$

$$0^{\text{SSM}} := (I_d, 0) \tag{23}$$

With this monoid structure, and the sequence $(a_i)_{i\in I} := ((A, x_i))_{i\in I} \in M^I$, the result of the parallel scan is $((A^i, \sum_{j=0}^{i} A^{i-j} x_j))_{i\in I}$. The complexity of the scan is less straightforward, but the complexity of each $+^{\text{SSM}}$ operation depends only on d, it is a constant of T, and therefore, the overall time complexity of the scan is $T_\infty = O(\log T)$ and $W = O(T)$.

Proposition 2 (SSM Complexity)**.** *Let* $(M, +) := (M^{\text{SSM}}, +^{\text{SSM}})$ *be the monoid defined by Equations (21) to (23). Let* $(a_i)_{i\in I} \in (M^{\text{SSM}})^I$.

$$W(\text{Scan}) = 2(T-1)(d^3 + d^2 + d) \tag{24}$$

$$T_\infty(\text{Scan}) = 2\log_2(T)(d^3 + d^2 + d) \tag{25}$$

Proof. The proof of Proposition 1 applies with the only difference that now, each $+^{\text{SSM}}$ operation has complexity d^3 for the matrix multiplication on the left, d^2 for the matrix-vector multiplication and d for the vector addition on the right. □

9.3 The problem of convolution

Moving from d dimensional vectors to images with c channels and $h \times w$ spatial dimension and from matrix multiplication to spatial convolution, we can rewrite our monoid structure as :

$$M^{\text{Conv}_1} := \left\{a = (K, I) \mid \exists k_1, k_2,\quad K \in \mathbb{R}^{k_1\times k_2\times c\times c},\ I \in \mathbb{R}^{h\times w\times c}\right\} \tag{26}$$

$$(K_a, I_a) +^{\text{Conv}_1} (K_b, I_b) := (K_b \circledast K_a, K_b * I_a + I_b) \tag{27}$$

$$0 := (I, 0) \tag{28}$$

where $\circledast$ is the convolution between two kernels and $*$ is the convolution between a kernel and an image:

$$\forall \begin{cases} x \in [\![0, h-1]\!] \\ y \in [\![0, w-1]\!] \\ c_{\text{out}} \in [\![1, c]\!] \end{cases}, (K * I)_{x,y,c_{\text{out}}} = \sum_{u=0}^{k_1-1}\sum_{v=0}^{k_2-1}\sum_{i=1}^{c} K_{u,v,i,c_{\text{out}}} I_{(x-u,y-v)\%(h,w),i}$$

$$\forall \begin{cases} u \in [\![0, k_1+l_1-2]\!] \\ v \in [\![0, k_2+l_2-2]\!] \\ c_{\text{in}}, c_{\text{out}} \in [\![1, c]\!] \end{cases}, (L \circledast K)_{u,v,c_{\text{in}},c_{\text{out}}} = \sum_{p=0}^{l_1-1}\sum_{q=0}^{l_2-1}\sum_{c_k=1}^{c} L_{p,q,c_k,c_{\text{out}}} K_{u-p,v-q,c_{\text{in}},c_k},\ K \text{ being zero-padded}$$

with $L \in \mathbb{R}^{l_1\times l_2\times c\times c}$, $K \in \mathbb{R}^{k_1\times k_2\times c\times c}$ and $I \in \mathbb{R}^{h\times w\times c}$

With this structure, the key aspect is that the spatial size of the kernel resulting from $\circledast$ is the sum of the sizes of the input kernels. Therefore, the parallel scan algorithm operates on larger and larger kernel sizes, with sizes growing exponentially with the depth in the tree structure. The complexity of our $+$ operator is thus no more constant as it depends on the depth. The bottleneck is at the very last layer of the upward scan, where kernel size is $O(Tk)$, with k the size of the original kernel K, resulting in a single $*$ operation whose complexity is in $O(T^2)$. This bottleneck is illustrated in Figure 8.

Proposition 3 (Spatial ConvSSM Complexity)**.** *Let* $(M, +) := (M^{\text{Conv}_1}, +^{\text{Conv}_1})$ *be the monoid defined by Equations (21) to (23). Let* $(a_i)_{i\in I} \in (M^{\text{Conv}_1})^I$.

$$W = O(T^2) \tag{29}$$

$$T_\infty = O(W) \tag{30}$$

Proof. The proof of Proposition 1 applies with the major difference that on the upward scan, at level l, each $+^{\text{Conv}_1}$ operation has complexity $(k+(2^l-1)(k-1)) \times (k+(2^l-1)(k-1)) \times c^3 = O((2^l)^2 \times k^2 \times c^3)$ for the kernel-kernel convolution on the left, $O(h \times w \times (2^l)^2 \times k^2 \times c^2)$ for the kernel-image convolution and $h \times w \times c$ for the image addition on the right. The same goes on for the downward scan.

Thus, Equations (19) and (20) become :

$$W = O(2 \sum_{l=0}^{\log_2(T)-1} 2^{\log_2(T)-(l+1)} \times \left[(2^l)^2 \times k^2 \times c^3 + h \times w \times (2^l)^2 \times k^2 \times c^2\right]) \tag{31}$$

$$= O(T^2 \times k^2 \times c^3) + O(T^2 \times h \times w \times k^2 \times c^2) \tag{32}$$

$$T_\infty = O(2 \sum_{l=0}^{\log_2(T)-1} \left[(2^l)^2 \times k^2 \times c^3 + h \times w \times (2^l)^2 \times k^2 \times c^2\right]) \tag{33}$$

$$= O(W) \tag{34}$$

□

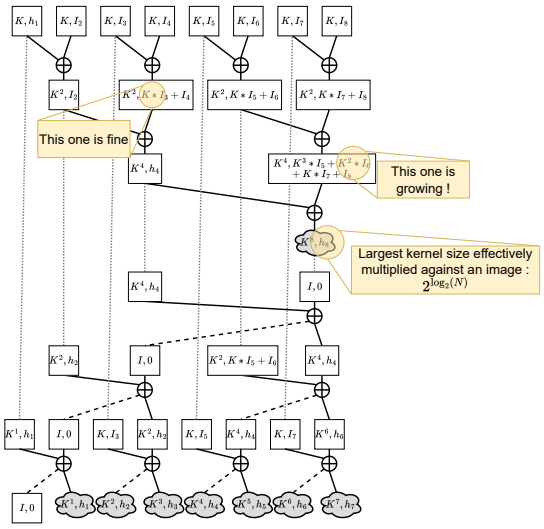

Figure 8: Parallel scan applied to the monoid structure defined for spatial convolution SSM. Highlighted is the core of the issue : growing kernels lead to a bottleneck where a single operation has $O(T^2)$ complexity.

9.4 How spectral domain solves the issue

To dispell the curse of growing kernels we move to the spectral domain. In this domain, our monoid becomes:

$$M^{\text{Conv}_2} := \left\{ a = (\hat{K}, \hat{I}) \;\middle|\; \exists (K, I) \in M^{\text{Conv}_1}, \begin{cases} \hat{K} = \text{FFT}(K_{\text{padded}}) \in \mathbb{C}^{h \times w \times c \times c} \\ \hat{I} = \text{FFT}(I) \in \mathbb{C}^{h \times w \times c} \end{cases} \right\} \tag{35}$$

$$(\hat{K}_a, \hat{I}_a) +^{\text{Conv}_2} (\hat{K}_b, \hat{I}_b) := (\hat{K}_b \odot \hat{K}_a, \hat{K}_b \cdot \hat{I}_a + \hat{I}_b) \tag{36}$$

$$0 := (I, 0) \tag{37}$$

with $\odot$ the element-wise matrix multiplication and $\cdot$ the element-wise matrix-vector multiplication. With this operator, the kernel grows no more, the cost is again a constant of time and depth, depending only on h, w and c. We are thus back to a time complexity for the parallel scan of $O(\log T)$.

Proposition 4 (Spectral ConvSSM Complexity)**.** *Let* $(M, +) := (M^{\text{Conv}_2}, +^{\text{Conv}_2})$ *be the monoid defined by Equations (35) to (37). Let* $(a_i)_{i \in I} \in (M^{\text{Conv}_2})^I$.

$$W = O(T) \tag{38}$$
$$T_\infty = O(\log_2 T) \tag{39}$$

Proof. The proof of Proposition 1 applies with the only difference that now, each $+^{\text{Conv}_2}$ operation has complexity $h \times w \times c^3$ for the kernel-kernel multiplication on the left, $h \times w \times c^2$ for the kernel-image multiplication and $h \times w \times c$ for the image addition on the right. This gives :

$$W = O(T \times h \times w \times c^3) \tag{40}$$
$$T_\infty = O(\log_2 T \times h \times w \times c^3) \tag{41}$$

□

9.5 Taking down the constant

Now that we have solved the main issue of complexity explosion with time, we tackle the problem of the constant. As mentioned in Proposition 2, as is, the work complexity is $O(T)$ with a constant that is $O(d^3)$ due to matrix multiplications. **?** proposed to reparameterise the recurrence matrix A to be normal plus low rank (NPLR) in order to reduce this constant from $O(d^3)$ to $O(d)$. The complexity gain comes from the change of basis being absorbed outside of the recurrence such that the recurrence can operate on diagonalized matrices. Note that their approach to parallelization through time uses the convolution view of the SSM, not the parallel scan as proposed by **?**. In our case, this $O(c^3)$ is even more prohibitive as it is repeated over each pixel position, with the total constant being $O(k^2c^3 + hwk^2c^2)$ in the spatial domain, and $O(hwc^3)$ in the spectral domain.

In spatial domain, this parameterization translates to having a shared change of basis V for each $K_{x,y}$. This way, V can be factored our ot the kernel such that the resulting kernel follows : $\forall x, y, \mathcal{K}_{x,y} = \Lambda_{x,y} + P_{x,y}Q_{x,y}^\top$, where

- $\Lambda_{x,y}$ are block diagonal with blocks of rank at most 2 in real domain, and diagonal with eigenvalues of modulus 1 in complex domain. With the complex view, $\mathcal{K}$ is diagonal plus low rank (DPLR).
- $P_{x,y}, Q_{x,y} \in \mathbb{R}^{d\times r}$ are low rank matrices, with rank r

In spectral domain, and after proper zero padding, such a parameterization yields :

$$\begin{aligned}
\forall w_1, w_2, \widehat{\mathcal{K}}_{w_1,w_2} &= \sum_{x,y} \alpha^{x,y}_{w_1,w_2} \mathcal{K}_{x,y} \\
&= \sum_{x,y} \alpha^{x,y}_{w_1,w_2} \left(\Lambda_{x,y} + P_{x,y}Q_{x,y}^\top\right) \\
&= \sum_{x,y} \alpha^{x,y}_{w_1,w_2} \Lambda_{x,y} + \sum_{x,y} \alpha^{x,y}_{w_1,w_2} P_{x,y}Q_{x,y}^\top
\end{aligned}$$

where $\alpha^{x,y}_{w_1,w_2} = e^{-2\pi i(\frac{w_1 x}{h} + \frac{w_2 y}{w})}$. There are a couple issues with this :

- The diagonal terms in spatial domain yield diagonal terms in spectral domain. However, these do not have modulus 1, and thus are not stable.
- The spectral term coming from the low rank terms is not low rank anymore : it now has rank up to k^2r instead of r. This issue can be solved by fixing $P_{x,y} = p,\ Q_{x,y} = q$ to be constants. This way, the associated spectral term is low rank.

We can circumvent both issues by parameterizing directly in the spectral domain. The tradeoff is that now,

- we are bound to a fixed resolution,
- the underlying spatial kernel has arbitrary support.

- the number of parameters in spectral domain is $hwc(2r+1)$ compared to $k^2c(2r+1)$ in spatial domain.

As **?** showed that parameterization of CNNs in spectral domain works just fine, we leave the question of finding a parameterization directly in pixel space to future work. Additionally, **?** showed that parameterizing the recurrence through a simple normal matrix is as expressive as the NPLR case. We thus take $r = 0$ and drop the low rank terms.

10 Towards Feedback Connections

In order to make lateral connections parallelisable, the SSM framework requires the constraint of linear recurrence. Applying the same constraint to feedback connections implies making the feed forward pathway linear as well, which is quite limiting. Indeed, with linear feedforwards and linear recurrence, the model becomes entirely linear and can be replaced by a single linear layer. A natural way to alleviate this issue is through linear skip connections. This creates many parallel feedforward pathways, one of which involves only the linear skip connections. The feedback pathway can then project from this linear skip connection pathway back to the lower layer, making it linear and parallelisable, while maintaining non linearity in the overall feedforward. This whole feedback pathway can then be collapsed into a single linear recurrence exactly as in the SSM framework. This idea is illustrated in **??**. Thus, linear recurrent layers give at once efficient parallelisable lateral connections, and efficient parallelisable feedback connections under the constraint that the feedback projects from linear skip connections. Parallelisable non linear recurrence and feedback are thus nothing but the very same problem, and solving this problem is an exciting avenue for future work.

In the context of biological neural circuits, this constraint finds a natural conterpart. As mentioned in Section 4, feedback connections are known to typically follow fast magnocellular pathways, while full feedforward pathways typically include slower parvocellular pathways. Thus, the constraint that feedback connections project from linear skip connections aligns well with the known biology of the visual system, providing a biologically plausible framework for modeling feedback in neural circuits. [Gregoire]: the above might be too much ? Additionally, in the visual cortex, feedback connections provide rapid, long range but coarse top-down modulation, while lateral connections are finer but slower and locally constrained. This finds a perfect equivalent in our convolutional framework, where feedback connections can be modelled as convolutions with large kernels, while lateral connections are modelled as convolutions with small kernels. Large kernels are typically computationally expensive, but since we work in spectral domain, they come with zero additional cost compared to smaller ones, and even allow for more efficient parameterisation directly in spectral domain without the need to go back and forth to spatial domain to impose kernel size constraints. Small kernels for lateral connections can also be modeled, and separately from their larger counterparts, allowing for more efficient parameterisation and interpretability. Indeed, **?** describes a way to parameterise orthogonal convolutions in spatial domain, allowing for a parameterisation of our discretized operator (Equation (11a)). [Gregoire]: Future work is needed to (i) make this parameterisation more expressive (cf discussion with thibaut), (ii) parameterise the continuous time operator, which does not need to be orthogonal but to have eigenvalues with negative real part for stability.

For biological plausibility or for interpretability sake, these two parameterisations can be separated into two consecutive ConvSSM layers, though this comes at some computational cost as both layers could be merged into a single one for efficiency. [Gregoire]: TODO : make ablation experiment to see if we gain in practice from separating lateral and feedback connections or if this does not help at all and a single "feedback" layer is enough.

11 Further Parameterizations

log domain (GOOM) / multiplicative behavior (might not need log domain if well parameterized, but log domain will help anyway) : circuit : F can include multiplicative or divisive interactions and modulations.

Ullman, S. (1984). Visual routines. Cognition, 18(1–3), 97–159. DOI: 10.1016/0010-0277(84)90023-4.

12 Experiments

Baselines and SOTA. SOTA : (h)Gru, ConvS4, ViT, ConvNeXt, etc. baselines : CNN ?

Classification. imagenet and compagnie

Visual Reasoning. static : pathfinder, cABC, 3D-PC, planko. Dynamic : feature tracker, pathtracker. Interpretability : attribution vs time, representation alignment, dynamics (attractors, fixed points, etc.), accuracy vs time / accuracy vs convergence to fixed points if any. Graal : pathfinder : have a window of attention moving along the path through time.

Self-Supervised Learning. dino like ssl, video ssl, predictive coding... interpretability : optical flow, motion prediction, modulation of activation and attribution maps in static areas, representation alignment with other sota ssl models.

13 Scaling Laws

how does performance scale with model size, data size, compute, etc. ?

14 Foundational ConvSSM

toy : visual reasoning tasks, generalisation / fine tuning on a held out task.

large scale : ssl then evaluate on downstream tasks.

15 Quantization

16 Discussion

...

Analyzing Multimodal Models

Part Outline

This part is independent on the previous one. It focuses on integrating several unimodal circuits into a single multimodal one. Its goal is not to introduce new architectures, but to analyse existing multimodal models through the lens of neural circuits, using interpretability tools.

17 Related Work

interpretability, multimodality, etc.

18 The Phenomenology of Multimodal Integration in LMMs

18.1 Framework Formulation

cf je sais plus quel overleaf.

18.2 Cycle Consistent IsoEnergy SAEs

Rewrite P_{tr}, P_{cont}, P_{dcy}, and P_{cy} from rufin et al., say that P_{tr} and P_{cont} rely on patching pairs for translation and alignment, which may not exist, but if the geometry aligns and arise from shared dictionaries, replacing them with IsoEnergy constraints should work. Define P_{iso} accordingly. Link to dhimoila et al. Show that the method works in a case where we have access to actual cycle consistent losses (e.g. CLIP).

figures : ...

tables : metrics ...

18.3 Dynamics of Multimodal Integration Across Layers

figures : (i) similarity matrices T1(L) vs T2(L'), or similarly with images : T1 is the distribution of text tokens without context, T2 is the distribution of text tokens with image context. If there is significant shift from T1 to T2, it means that image context is integrated into text tokens. (ii) similarity matrices T1(L) vs T1(L') : if significant shift, it means that the model is changing the geometry of the representation space across layers. This should be obvious in T as input tokens are raw and uncontextualised. The interesting case is for I as "input" tokens are already fully refined by the visual system. (iii) similarity matrices T1(L) vs I1(L') : this needs matching pairs or isoenergy constraints to work... enter CCIE SAEs. Expect image and text to get closer across layers maybe until the last few layers where text needs to collapse for token prediction. Under assumption 2 : wasserstein between distributions. Possibly include translation / scale invariance by normalising representations first. Under assumption 3 : SAE based metrics.

[Gregoire]: TODO : normalise CMAP in the range of the metric. Currently, plots that range between 0 and 1e-11 will have as bright of a color as plots that have the full range 0-1 or 0-2. [Gregoire]: have different color maps for different metrics of distribution alignment.

(a) Text distribution (b) Image distribution

Figure 9: Similarity matrices showing the evolution of text (*left*) and image (*right*) token distributions across layers in a multimodal model. The evolution of text token across layer is standard, and shows that contextualisation and refinment occurs across layers, starting from uncontextualised raw tokens. Visual token evolution is of particular interest here, as visual tokens are already fully refined by the vision backbone. Any significant change in their geometry across layers indicates further refinement or contextualisation performed by the language backbone. [Gregoire]: TODO : interpret the actual results. The metric reported here is **TODO**. See **??** for other metrics.

(a) Text integration (b) Image integration

Figure 10: Multimodal integration dynamics across layers. Contrary to Fig. 9, which focused on the evolution of the geometry of each modality independently, here we focus on the effect of multimodal integration on this geometry. Specifically, we compare the distribution of text tokens without image context to the distribution of text tokens with image context (*left*), and similarly for image tokens (*right*). The diagonal of this distance matrices is of particular interest, as the off diagonal elements are mainly driven by layer-wise drift of the geometry rather than multimodal integration effects. [Gregoire]: TODO : interpret the actual results.

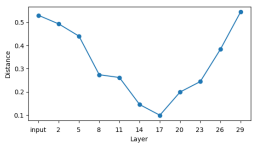

Figure 11: Geometric similarity between text and image token distributions across layers. This figure shows how the geometry of text and image token distributions align across layers in a multimodal model. [Gregoire]: TODO : interpret the actual results. Both distributions seem to converge to a shared latent geometry. If the previous figures shows that image geometry is roughly constant, we should see stripes of equal alignment

19 Interpretability of Multimodal Integration in LMMs

19.1 Identifying Multimodal Neurons

e.g. a ”blue” neuron that responds to both the concept of ”blue” in text and images. Use SAEs for automated identification, e.g. the whole Gamma subspace if that works, or probes for specific concepts. causal intervention : fiddle with these neurons and check for predictable effect on both vision and language token fiddling.

19.2 Visual Integration Circuits

Are there dedicated circuits that move information from vision tokens to language tokens ? Or does the visual token just align with latent geometry and the information retrieval follows arbitrary pre existing language circuits ? Start with task specific then move on to general ones. causal interventions : 1st case : ablate these circuits, check that text only capabilities are preserved but vision integration is lost. 2nd case : since these are not dedicated multimodal integration pathways but recycled unimodal language circuits leveraged through representational alignment, ablating them should break both vision integration and text only capabilities. This gives two hypotheses that can be tested experimentally.

19.3 On the Binding Problem

there might not be a binding problem at all... in LMM, it’s either trivially solved when there is no more than one object of interest per token, or unsolved due to the vision backbone, not due to multimodal integration per se.

20 Discussion

...

Towards Multimodal Neural Circuits

Part Outline

Combining insights from parts II and III, we propose new multimodal architectures inspired by neural circuits in light of what we learned. (speculative part for now)

21 Introduction

multimodal, embodied agents, neural circuits, patati patata...

22 Related Work

...

23 …

...

Universal Neural Circuits

Part Outline

We propose the notion of universal neural circuits (UNC), akin to universal turing machines, that can simulate any other neural circuit under some constraints. This could serve as a foundation model for brain science.

24 Introduction

EEG, MEG, sEEG, etc. Foundation models for brain science.

25 Related Work

...

26 …

...

Appendix

Table of Contents

27 State Space Models. **16**

27 State Space Models.

Neural circuits rely heavilly on recurrent connections, key to model long range dependencies and working memory. However, training recurrent neural networks is notoriously difficult due to vanishing and exploding gradients on the optimisation side, and to sequentiality incompatible with modern hardware acceleration on the computational side. Recently, state space models (SSMs) have emerged as a powerful class of models to tackle these issues. The core idea of SSMs is to model the recurrent dynamics as a linear system, allowing to leverage linear algebra tools to parallelise training and inference, while maintaining the ability to model long range dependencies through carefully designed dynamics. See Section 8 for a formal definition and analysis of the stability of these systems. See Section 9 for a detailed analysis of their computational complexity in light of modern hardware acceleration. SSMs have been successfully applied to various domains, including natural language processing (**?**), computer vision (**?**), and time series analysis (**?**).